

Padrões de Projeto (Adapter)

QXD0068 - Reuso de Software

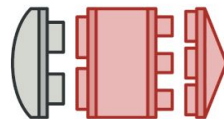
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Adapter
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



Adapter

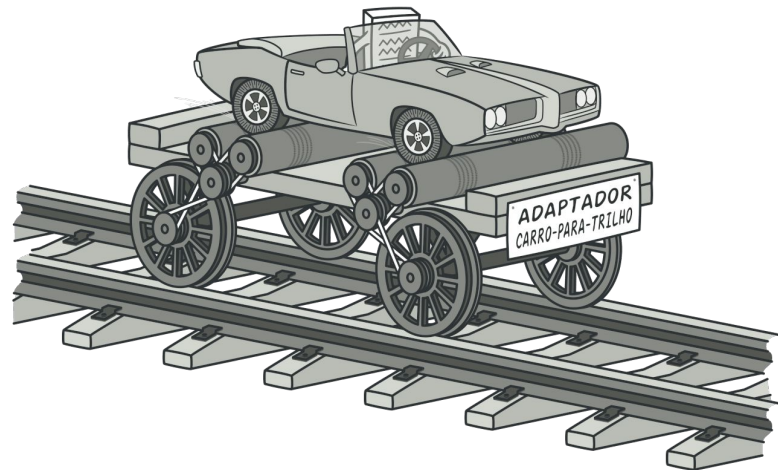
Permite a colaboração de objetos de interfaces incompatíveis.

Introdução

- O Adapter (também conhecido como Adaptador ou Wrapper) é um padrão de projeto estrutural que permite que objetos com interfaces incompatíveis colaborem entre si.
- Ele atua como um “tradutor” entre duas classes que não poderiam se comunicar diretamente.

Padrão Adapter

- O padrão resolve problemas de incompatibilidade de interfaces, permitindo reutilizar código existente sem modificá-lo.
- Ele converte a interface de uma classe em outra interface esperada pelos clientes.

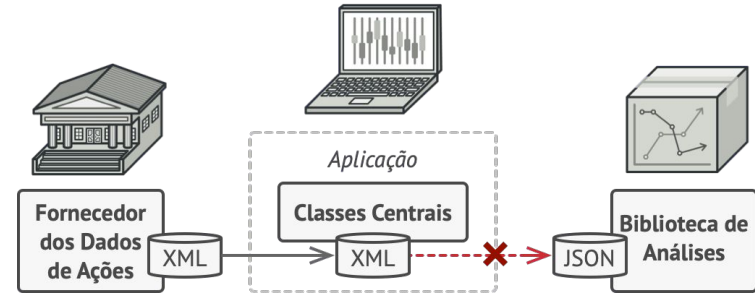


Problema

- Imagine que você está criando uma aplicação de monitoramento do mercado de ações da bolsa. A aplicação baixa os dados das ações de múltiplas fontes em formato XML e então mostra gráficos e diagramas para o usuário. Em algum ponto, você decide melhorar a aplicação ao integrar uma biblioteca de análise de terceiros. Mas aqui está a pegadinha: a biblioteca só trabalha com dados em formato JSON.
- A estrutura da aplicação antes da integração com a biblioteca de análises. Você não pode usar a biblioteca “como ela está” porque ela espera os dados em um formato que é incompatível com sua aplicação.
- Você poderia mudar a biblioteca para que ela funcione com XML. Contudo, isso pode quebrar algum código existente que depende da biblioteca. E pior, você pode não ter acesso ao código fonte da biblioteca para começar de conversa, fazendo dessa abordagem uma tarefa impossível.

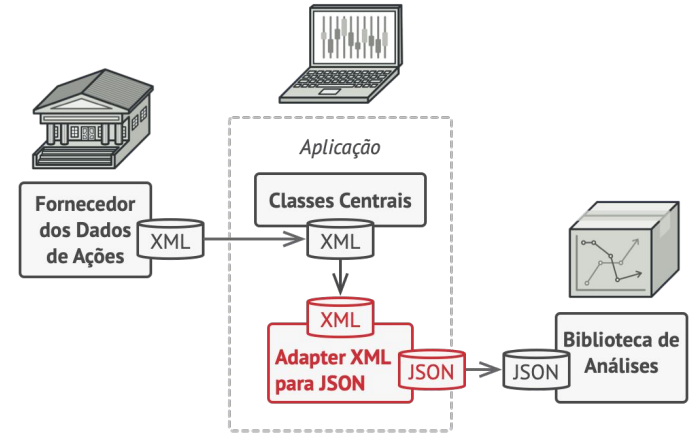
Problema

- A aplicação usa XML para dados de ações.
- A nova biblioteca aceita apenas JSON.
- Incompatibilidade de formatos impede a integração direta.
- Não é possível alterar a biblioteca (sem acesso ao código-fonte).
- É preciso uma solução intermediária para converter XML $\rightarrow$ JSON.



Solução

- Cria-se um adaptador — um objeto que encobre o componente incompatível e traduz os dados e chamadas entre as interfaces.
- **O adaptador:**
 - Implementa a interface esperada pelo cliente.
 - Contém internamente uma instância da classe de serviço.
 - Traduz as chamadas e formatos de dados de um lado para o outro.
- **Pode ser implementado:**
 - Por composição (adaptador de objeto).
 - Por herança múltipla (adaptador de classe, quando suportado).

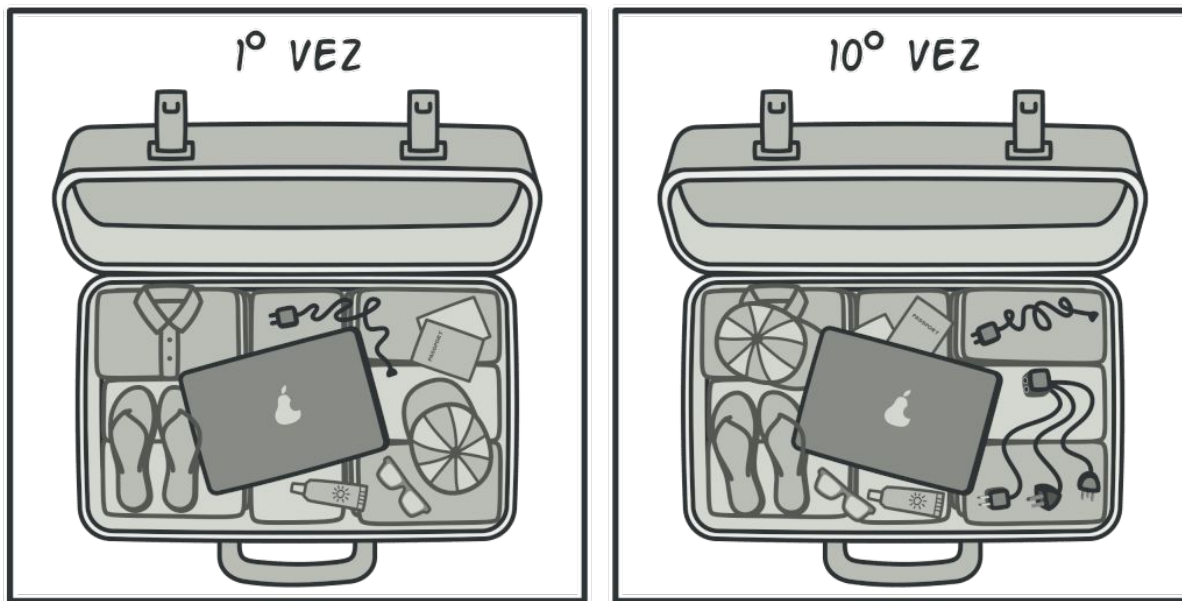


Analogia do Mundo Real

- Assim como um adaptador de tomada, que permite conectar um plugue brasileiro a uma tomada europeia, o padrão Adapter permite conectar classes com “formatos” diferentes (interfaces incompatíveis).

Analogia do Mundo Real

VIAJANDO PARA O EXTERIOR



Estrutura do Padrão

- **Componentes principais:**
 - Cliente: o sistema existente que precisa usar o serviço.
 - Interface do Cliente: define o contrato que o cliente espera.
 - Serviço: classe existente (muitas vezes de terceiros) com interface incompatível.
 - Adaptador: implementa a interface do cliente e faz a conversão de chamadas para o serviço.

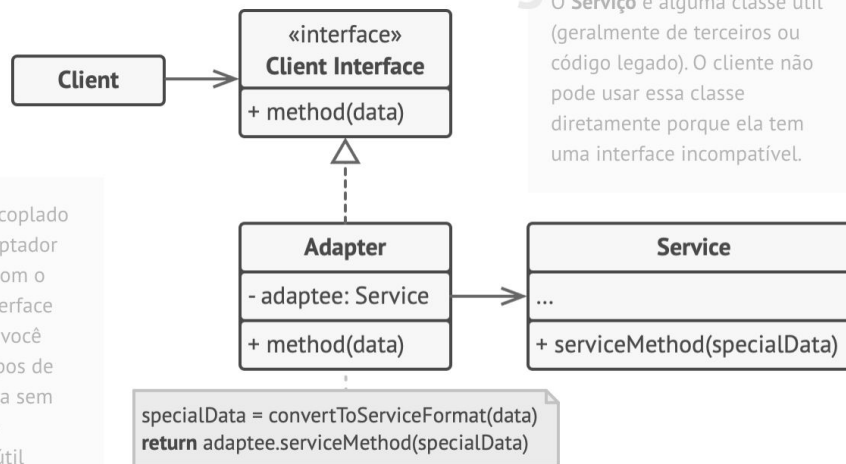
- **Tipos:**
 - Adaptador de Objeto: usa composição (mais flexível e comum).
 - Adaptador de Classe: usa herança múltipla (limitado a linguagens que a suportam).

1 O **Cliente** é uma classe que contém a lógica de negócio do programa existente.

2 A **Interface do Cliente** descreve um protocolo que outras classes devem seguir para ser capaz de colaborar com o código cliente.

3 O **Serviço** é alguma classe útil (geralmente de terceiros ou código legado). O cliente não pode usar essa classe diretamente porque ela tem uma interface incompatível.

5 O código cliente não é acoplado à classe concreta do adaptador desde que ele trabalhe com o adaptador através da interface do cliente. Graças a isso, você pode introduzir novos tipos de adaptadores no programa sem quebrar o código cliente existente. Isso pode ser útil quando a interface de uma classe de serviço é mudada ou substituída: você pode apenas criar uma nova classe adaptador sem mudar o código cliente.



4 O **Adaptador** é uma classe que é capaz de trabalhar tanto com o cliente quanto o serviço: ela implementa a interface do cliente enquanto encobre o objeto do serviço. O adaptador recebe chamadas do cliente através da interface do cliente e as traduz em chamadas para o objeto encobrido do serviço em um formato que ele possa entender.

```
// Interface esperada pelo cliente
interface MediaPlayer {
    fun play(fileType: String, fileName: String)
}

// Classe existente (incompatível)
class AdvancedPlayer {
    fun playMp4(fileName: String) = println("Reproduzindo arquivo MP4: $fileName")
}

// Adaptador que traduz a interface
class MediaAdapter(private val advancedPlayer: AdvancedPlayer) : MediaPlayer {
    override fun play(fileType: String, fileName: String) {
        if (fileType == "mp4") advancedPlayer.playMp4(fileName)
        else println("Formato não suportado")
    }
}

// Cliente usando o adaptador
fun main() {
    val adapter = MediaAdapter(AdvancedPlayer())
    adapter.play("mp4", "video_exemplo.mp4")
}
```

Exemplo



```
class XMLData:
    def get_xml(self):
        return "<dados>...</dados>"

class JSONAnalyzer:
    def analyze(self, json_data):
        print(f"Analisando dados JSON: {json_data}")

# Adaptador
class XMLtoJSONAdapter:
    def __init__(self, xml_data):
        self.xml_data = xml_data

    def get_json(self):
        xml = self.xml_data.get_xml()
        return '{"dados": "..."}' # conversão fictícia

xml = XMLData()
adapter = XMLtoJSONAdapter(xml)
analyzer = JSONAnalyzer()
analyzer.analyze(adapter.get_json())
```

Exemplo



Aplicabilidade

- **Use o Adapter quando:**

- Você precisa usar uma classe existente com interface incompatível.
- Deseja reutilizar código legado ou de terceiros sem modificá-lo.
- Precisa adicionar funcionalidade comum a várias subclasses sem duplicar código (semelhante ao Decorator).

Prós e Contras

- **Vantagens**

- Responsabilidade única: separa a lógica de conversão da lógica principal.
- Aberto/fechado: novos adaptadores podem ser adicionados sem alterar o código cliente.

- **Desvantagens**

- Aumenta a complexidade do sistema com novas classes e interfaces.
- Em alguns casos, pode ser mais simples alterar a classe original (quando possível).

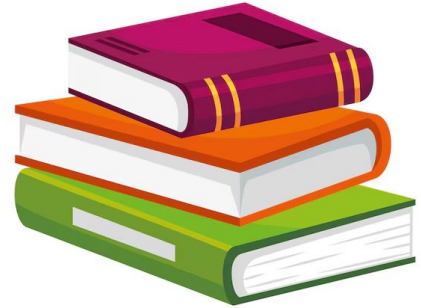
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

